Applied Bioinformatics 2026

Week09 - Generate a VCF file

Introduction

Generate a VCF file

In the Generate a BAM file assignment you aligned reads to your genome and created a BAM file.

You will be asked to call variants from that alignment and create a VCF file. You can start with the previous Makefile and insert additionalsteps to call variants.

Study the Variant calling section before you start.

Call variants from the BAM file

1. Ensure that you get at least 10x coverage of the genome.
2. Write a Makefile that calls variants and creates a VCF file.
3. Run a statistics report on the VCF file.
4. Visualize the VCF file in IGV.
5. Write a README.md that a reviewer can follow.

In the case that you get no variants, and all reads match perfectly find a different sample, perhaps a different strain, or a sample from an evolutionary close and similar organism.

What to put in the README

- How many variants were called?
- What kinds of variants are present?
- Which calls look like true variants, and which look like errors?
- Are the calls supported by the alignments?

Include the commands needed to run the Makefile and a screenshot of the VCF file in IGV.

Analysis

Run the makefile to see the options:

```
make
```

```
Applied Bioinformatics 2026
Generate a BAM file and extend the workflow to create a VCF file

This Makefile downloads a reference genome and FASTQ reads,
indexes the genome with BWA, aligns the reads to the reference,
```

```
creates a sorted and indexed BAM file, and extends the workflow
by calling genomic variants and creating a VCF file with bcftools.

Available commands:
  make all      Run the complete workflow
  make genome   Download the reference genome
  make fastq    Download FASTQ reads from SRA
  make index    Index the reference genome with BWA
  make align    Align reads and create a BAM file
  make vcf      Call variants and create a VCF file
  make clean    Remove generated files

Configurable variables:

GENOME_URL=https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/025/998/455/GCF_025
998455.1_ASM2599845v1/GCF_025998455.1_ASM2599845v1_genomic.fna.gz
  GENOME_NAME=CHC155
  SRR=SRR27606652
  N=40000
  SAMPLE_NAME=412F
```

The contents of the makefile:

```
# Applied Bioinformatics 2026
# Generate a BAM file and extend the workflow to call genomic variants.
#
# This Makefile downloads a reference genome and FASTQ reads,
# indexes the genome with BWA, aligns the reads to the reference,
# creates a sorted and indexed BAM file, and extends the workflow
# by using bcftools to identify genomic variants and create a VCF file.

# URL of the reference genome
GENOME_URL =
https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/025/998/455/GCF_025998455.1_AS
M2599845v1/GCF_025998455.1_ASM2599845v1_genomic.fna.gz

# Name of the reference genome
GENOME_NAME = CHC155

# Genome fasta file
GENOME_FASTA = fasta/$(GENOME_NAME).fasta

# SRA accession number
SRR = SRR27606652

# Number of reads to download
N = 40000

# Fastq files
FASTQ_1 = fastq/$(SRR)_1.fastq
FASTQ_2 = fastq/$(SRR)_2.fastq
```

```
# Sample name
SAMPLE_NAME = 412F

# BAM file derived from the sample name
BAM = bam/$(SAMPLE_NAME).bam

# VCF file derived from the sample name
VCF = vcf/$(SAMPLE_NAME).vcf.gz

# Annotation parameters for bcftools mpileup
ANN = -d 100 --annotate
'INFO/AD,FORMAT/DP,FORMAT/AD,FORMAT/ADF,FORMAT/ADR,FORMAT/SP'

# Original diploid setting from the example.
# CALL = --ploidy 2 --annotate 'FORMAT/GQ'

# H. pylori is a haploid bacterium, so variants should be called
# using ploidy 1 rather than the diploid setting used in the example.
CALL = --ploidy 1 --annotate 'FORMAT/GQ'

# BWA index sentinel
INDEX = $(GENOME_FASTA).bwt

# FASTA index for bcftools
FASTA_INDEX = $(GENOME_FASTA).fai

SHELL = bash
.SHELLFLAGS = -eu -o pipefail -c
.DELETE_ON_ERROR:
MAKEFLAGS += --warn-undefined-variables
MAKEFLAGS += --no-builtin-rules

# --- All definitions above this line. All code below this line ---

# Show available commands and variables.
help:
    @echo "Applied Bioinformatics 2026"
    @echo "Generate a BAM file and extend the workflow to create a VCF
file"
    @echo ""
    @echo "This Makefile downloads a reference genome and FASTQ reads,"
    @echo "indexes the genome with BWA, aligns the reads to the reference,"
    @echo "creates a sorted and indexed BAM file, and extends the workflow"
    @echo "by calling genomic variants and creating a VCF file with
bcftools."
    @echo ""
    @echo "Available commands:"
    @echo "  make all      Run the complete workflow"
    @echo "  make genome   Download the reference genome"
    @echo "  make fastq    Download FASTQ reads from SRA"
    @echo "  make index    Index the reference genome with BWA"
    @echo "  make align    Align reads and create a BAM file"
    @echo "  make vcf      Call variants and create a VCF file"
    @echo "  make clean    Remove generated files"
```

```
    @echo ""
    @echo "Configurable variables:"
    @echo "  GENOME_URL=$(GENOME_URL)"
    @echo "  GENOME_NAME=$(GENOME_NAME)"
    @echo "  SRR=$(SRR)"
    @echo "  N=$(N)"
    @echo "  SAMPLE_NAME=$(SAMPLE_NAME)"

# Run all steps.
all: genome fastq index align vcf

# Download the genome.
genome: $(GENOME_FASTA)

$(GENOME_FASTA):
    mkdir -p $(dir $@)
    curl -fsSL $(GENOME_URL) | gunzip > $@
    seqkit stats $@

# Download FASTQ files from SRA.
fastq: $(FASTQ_1) $(FASTQ_2)

$(FASTQ_1) $(FASTQ_2):
    mkdir -p $(dir $@)
    fastq-dump -X $(N) --split-files -O $(dir $@) $(SRR)
    seqkit stats $(FASTQ_1) $(FASTQ_2)

# Index the genome.
index: $(INDEX)

$(INDEX): $(GENOME_FASTA)
    bwa index $<

# Align the FASTQ reads to the reference genome.
align: $(BAM)

$(BAM): $(INDEX) $(FASTQ_1) $(FASTQ_2)
    mkdir -p $(dir $@)
    bwa mem $(GENOME_FASTA) $(FASTQ_1) $(FASTQ_2) | samtools sort -o $@
    samtools index $@
    samtools flagstat $@

# Index the FASTA for variant calling.
$(FASTA_INDEX): $(GENOME_FASTA)
    samtools faidx $<

# Call variants and create a compressed, indexed VCF file.
vcf: $(VCF)

$(VCF): $(BAM) $(FASTA_INDEX)
    mkdir -p $(dir $@)
    bcftools mpileup $(ANN) -O u -f $(GENOME_FASTA) $(BAM) | \
        bcftools call $(CALL) -mv -O u | \
        bcftools norm -f $(GENOME_FASTA) -d all -O u | \
```

```
        bcftools sort --write-index -O z -o $@

# Remove generated files.
clean:
    rm -rf fasta fastq bam vcf

.PHONY: help all genome fastq index align vcf clean
```

Run make -n all to see the steps involved:

```
make -n all
```

```
mkdir -p fasta/
curl -fsSL
https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/025/998/455/GCF_025998455.1_AS
M2599845v1/GCF_025998455.1_ASM2599845v1_genomic.fna.gz | gunzip >
fasta/CHC155.fasta
seqkit stats fasta/CHC155.fasta
mkdir -p fastq/
fastq-dump -X 40000 --split-files -O fastq/ SRR27606652
seqkit stats fastq/SRR27606652_1.fastq fastq/SRR27606652_2.fastq
mkdir -p fastq/
fastq-dump -X 40000 --split-files -O fastq/ SRR27606652
seqkit stats fastq/SRR27606652_1.fastq fastq/SRR27606652_2.fastq
bwa index fasta/CHC155.fasta
mkdir -p bam/
bwa mem fasta/CHC155.fasta fastq/SRR27606652_1.fastq
fastq/SRR27606652_2.fastq | samtools sort -o bam/412F.bam
samtools index bam/412F.bam
samtools flagstat bam/412F.bam
samtools faidx fasta/CHC155.fasta
mkdir -p vcf/
bcftools mpileup -d 100 --annotate
'INFO/AD,FORMAT/DP,FORMAT/AD,FORMAT/ADF,FORMAT/ADR,FORMAT/SP' -O u -f
fasta/CHC155.fasta bam/412F.bam | \
        bcftools call --ploidy 1 --annotate 'FORMAT/GQ' -mv -O u | \
        bcftools norm -f fasta/CHC155.fasta -d all -O u | \
        bcftools sort --write-index -O z -o vcf/412F.vcf.gz
(bioinfo)
```

Run the entire workflow:

```
make all
```

```
mkdir -p fasta/
curl -fsSL
https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/025/998/455/GCF_025998455.1_AS
M2599845v1/GCF_025998455.1_ASM2599845v1_genomic.fna.gz | gunzip >
fasta/CHC155.fasta
seqkit stats fasta/CHC155.fasta
file                format  type  num_seqs    sum_len    min_len    avg_len
max_len
fasta/CHC155.fasta  FASTA   DNA          1  1,696,601  1,696,601  1,696,601
1,696,601
mkdir -p fastq/
fastq-dump -X 40000 --split-files -O fastq/ SRR27606652
Read 40000 spots for SRR27606652
Written 40000 spots for SRR27606652
seqkit stats fastq/SRR27606652_1.fastq fastq/SRR27606652_2.fastq
processed files:  2 / 2 [======================================] ETA: 0s.
done
file                        format  type  num_seqs     sum_len  min_len
avg_len  max_len
fastq/SRR27606652_1.fastq  FASTQ   DNA     40,000  10,040,000      251
251      251
fastq/SRR27606652_2.fastq  FASTQ   DNA     40,000   9,120,000      228
228      228
bwa index fasta/CHC155.fasta
[bwa_index] Pack FASTA... 0.01 sec
[bwa_index] Construct BWT for the packed sequence...
[bwa_index] 0.35 seconds elapse.
[bwa_index] Update BWT... 0.01 sec
[bwa_index] Pack forward-only FASTA... 0.01 sec
[bwa_index] Construct SA from BWT and Occ... 0.18 sec
[main] Version: 0.7.19-r1273
[main] CMD: bwa index fasta/CHC155.fasta
[main] Real time: 0.814 sec; CPU: 0.553 sec
mkdir -p bam/
bwa mem fasta/CHC155.fasta fastq/SRR27606652_1.fastq
fastq/SRR27606652_2.fastq | samtools sort -o bam/412F.bam
[M::bwa_idx_load_from_disk] read 0 ALT contigs
[M::process] read 41754 sequences (10000083 bp)...
[M::process] read 38246 sequences (9159917 bp)...
[M::mem_pestat] # candidate unique pairs for (FF, FR, RF, RR): (0, 17026,
4, 0)
[M::mem_pestat] skip orientation FF as there are not enough pairs
[M::mem_pestat] analyzing insert size distribution for orientation FR...
[M::mem_pestat] (25, 50, 75) percentile: (334, 435, 524)
[M::mem_pestat] low and high boundaries for computing mean and std.dev: (1,
904)
[M::mem_pestat] mean and std.dev: (425.96, 136.60)
[M::mem_pestat] low and high boundaries for proper pairs: (1, 1094)
[M::mem_pestat] skip orientation RF as there are not enough pairs
[M::mem_pestat] skip orientation RR as there are not enough pairs
[M::mem_process_seqs] Processed 41754 reads in 9.878 CPU sec, 9.905 real
sec
```

```
[M::mem_pestat] # candidate unique pairs for (FF, FR, RF, RR): (0, 15580,
6, 0)
[M::mem_pestat] skip orientation FF as there are not enough pairs
[M::mem_pestat] analyzing insert size distribution for orientation FR...
[M::mem_pestat] (25, 50, 75) percentile: (337, 436, 525)
[M::mem_pestat] low and high boundaries for computing mean and std.dev: (1,
901)
[M::mem_pestat] mean and std.dev: (427.99, 135.64)
[M::mem_pestat] low and high boundaries for proper pairs: (1, 1089)
[M::mem_pestat] skip orientation RF as there are not enough pairs
[M::mem_pestat] skip orientation RR as there are not enough pairs
[M::mem_process_seqs] Processed 38246 reads in 9.153 CPU sec, 9.199 real
sec
[main] Version: 0.7.19-r1273
[main] CMD: bwa mem fasta/CHC155.fasta fastq/SRR27606652_1.fastq
fastq/SRR27606652_2.fastq
[main] Real time: 19.252 sec; CPU: 19.127 sec
samtools index bam/412F.bam
samtools flagstat bam/412F.bam
80951 + 0 in total (QC-passed reads + QC-failed reads)
80000 + 0 primary
0 + 0 secondary
951 + 0 supplementary
0 + 0 duplicates
0 + 0 primary duplicates
70802 + 0 mapped (87.46% : N/A)
69851 + 0 primary mapped (87.31% : N/A)
80000 + 0 paired in sequencing
40000 + 0 read1
40000 + 0 read2
67888 + 0 properly paired (84.86% : N/A)
68576 + 0 with itself and mate mapped
1275 + 0 singletons (1.59% : N/A)
0 + 0 with mate mapped to a different chr
0 + 0 with mate mapped to a different chr (mapQ>=5)
samtools faidx fasta/CHC155.fasta
mkdir -p vcf/
bcftools mpileup -d 100 --annotate
'INFO/AD,FORMAT/DP,FORMAT/AD,FORMAT/ADF,FORMAT/ADR,FORMAT/SP' -O u -f
fasta/CHC155.fasta bam/412F.bam | \
        bcftools call --ploidy 1 --annotate 'FORMAT/GQ' -mv -O u | \
        bcftools norm -f fasta/CHC155.fasta -d all -O u | \
        bcftools sort --write-index -O z -o vcf/412F.vcf.gz
[mpileup] 1 samples in 1 input files
Writing to /tmp/bcftools.Lsc0X3
[mpileup] maximum number of reads per input file set to -d 100
Note: The maximum per-sample depth with -d 100 is 100.0x
[W::bcf_hdr_check_sanity] MQ should be declared as Type=Float
[W::bcf_hdr_check_sanity] MQ should be declared as Type=Float
Lines   total/split/joined/realigned/mismatch_removed/dup_removed/skipped:
47575/0/0/487/0/0/0
Merging 1 temporary files
```

```
Done
Cleaning
```

Run a statistics report on the VCF file:

```
bcftools stats vcf/412F.vcf.gz > stats.txt
```

Results

How many variants were called?

What kinds of variants are present?

Get a summary of the stats.txt:

```
grep -E '^(SN|TSTV)' stats.txt
```

```
SN      0       number of samples:      1
SN      0       number of records:      47575
SN      0       number of no-ALTs:      0
SN      0       number of SNPs: 46860
SN      0       number of MNPs: 0
SN      0       number of indels:       715
SN      0       number of others:       0
SN      0       number of multiallelic sites:   0
SN      0       number of multiallelic SNP sites:       0
TSTV    0       38465   8395    4.58    38465   8395    4.58
```

Which calls look like true variants, and which look like errors?

Many of the variant calls appear to be true variants because the same base difference is visible in multiple aligned reads at the same genomic position. These appear in IGV as vertical columns of matching colored bases beneath corresponding calls in the VCF track. Calls that are supported by only one or a few reads, occur near the ends of reads, or are located in regions with lower or irregular coverage are less convincing and could represent sequencing or alignment errors.

Are the calls supported by the alignments?

Yes, many of the calls are supported by the alignments. The blue markers in the `412F.vcf.gz` track frequently line up with colored mismatch columns in the `412F.bam` reads below. This means the VCF calls correspond to repeated differences observed directly in the sequencing reads. Some positions show stronger support than others, so the alignments help distinguish well-supported variants from weaker or potentially erroneous calls.

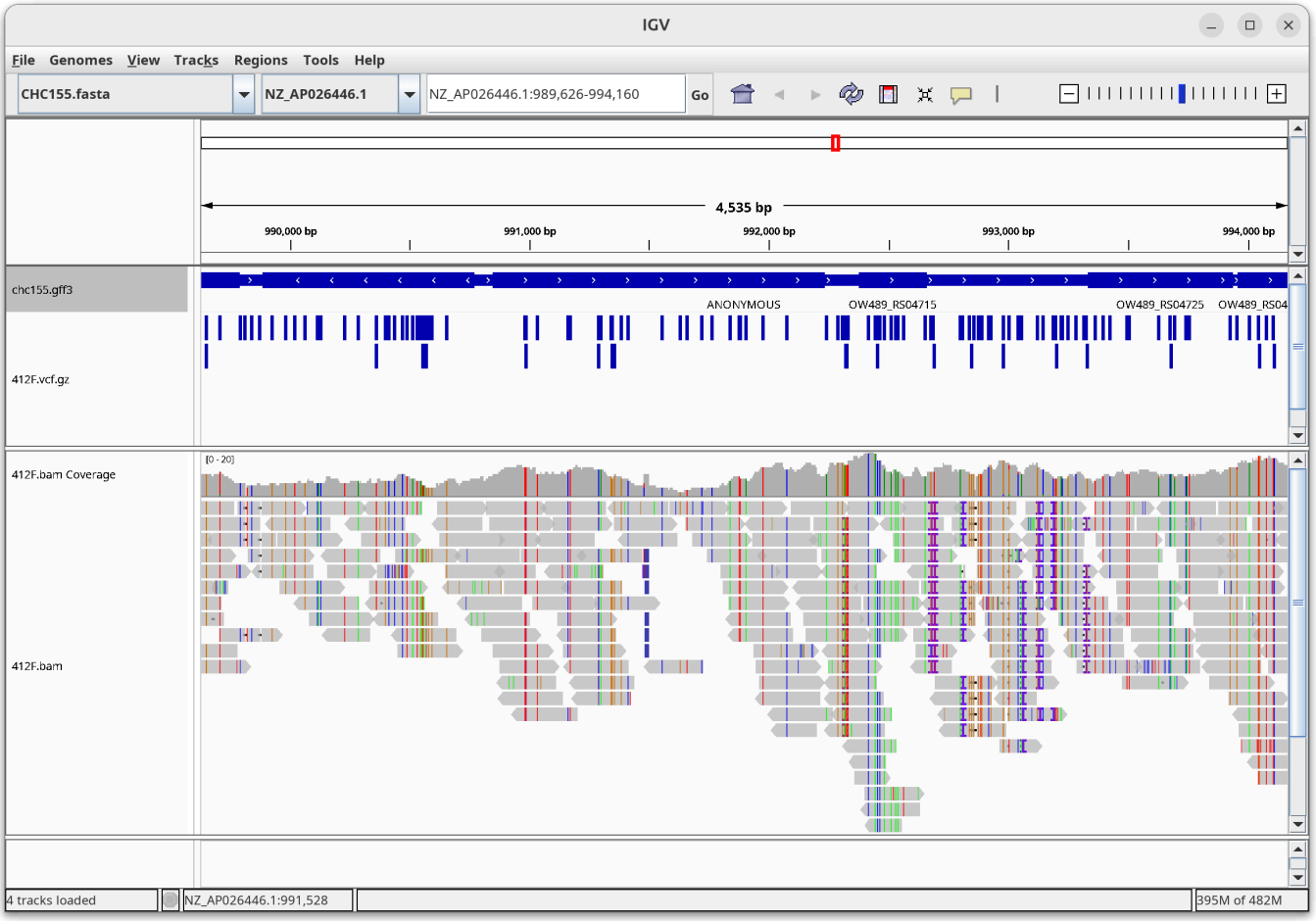


Figure: IGV view of the CHC155 reference genome with the 412F VCF and BAM tracks. Blue marks in the VCF track indicate called variants, while colored bases in the aligned reads show mismatches relative to the reference. Many variant calls align with vertical columns of the same colored base across multiple reads, providing visual support that these are true variants. Calls with weak or inconsistent read support are less certain and may represent sequencing or alignment errors.

Summary

In this exercise, we extended the existing *H. pylori* workflow from read alignment into genomic variant analysis. We used the 412F BAM file and the CHC155 reference genome to call variants with `bcftools`, corrected the variant-calling ploidy from diploid to haploid, generated and summarized a compressed VCF file, and found 47,575 variants consisting mainly of 46,860 SNPs and 715 indels. We also reviewed the variant statistics and examined the VCF and BAM tracks together in IGV, where many called variants were visibly supported by consistent base differences across multiple aligned reads, while weaker calls with limited or irregular read support were considered more likely to represent sequencing or alignment errors.